Write a fuzzer

The goal of this project is to write a *fuzzer* for a simple implementation of a *tar* extractor. [1]
This project must be done by **groups of max. 2 students**. Submitting alone is also accepted but should be avoided as much as possible.

The *tar* format

`tar` is a file format for file archives. It concatenates files and custom headers in a whole archive. The description of those headers is available at
https://www.gnu.org/software/tar/manual/html_node/Standard.html.
For this project, we use the **POSIX 1003.1-1990** format.
To create an archive corresponding to this format you can use

```
tar --posix --pax-option delete=".*" --pax-option delete="*time*" --no-xattrs --no-acl --no-selinux -c fichier1 fichier2 ... > archive.tar
```

Your fuzzer **may not** use this command.

To visualize the hexadecimal content of the archive, use the following command:

```
hexdump -C archive.tar
```

What is a fuzzer?

The *tar* extractor works correctly for input files matching the aforementioned specification. However, it crashes sometimes if the input file is not correctly formatted. In that case, it writes

```
*** The program has crashed ***
```

on stdout.
This is of course very dangerous. Imagine what could happen if such a vulnerable tool ran on a web server, for example on INGInious that would allow students to upload their code as tar archive during an exam.
Security experts sometimes use *fuzzing* tools to find vulnerabilities in programs. A fuzzer is a tool that generates input data with the goal of crashing the tested program. When such input data is found, it is saved so it can be analyzed later by security experts.
There are different types of fuzzers:

1. In the simplest form, a fuzzer generates purely random input files. Such a fuzzer is easy to write but quite inefficient: Most input files would probably not be accepted by the tested program because they have the wrong format.
2. *Mutation-based* fuzzers take a valid input file and modify it slightly, for example, by adding additional bytes at random places.
3. Generation-based fuzzers generate (almost) valid input files based on the knowledge of the input format. To find vulnerabilities in the tested program, they often test extreme cases (e.g. very large numbers in an input field, etc.).

Your job

Your job is to write a <u>generation-based fuzzer</u> for a *tar* extractor. The fuzzer should automatically generate input files (not mutate a given one) and check whether the extractor crashes. Input files that successfully crash the extractor are saved.

We give you 6 toy-example extractors that in the course VM in /home/student/fuzzer/. The extractors are executed by the following command:

---

1 The *tar* extractor is under MIT License (available at https://opensource.org/licenses/MIT) Copyright 2026 Tom Rousseaux, Ramin Sadre

```
./bugX archive.tar
```
where archive.tar is the tar file.

We are aware of at least six different ways to crash these toy-example extractors (one way by extractor), i.e. when the program writes the crash message.

Be aware that your fuzzer **will be tested on other extractors** than those we give you. It means that if you discover that an extractor crashes for `typeflag=\x90`, you cannot hardcode this value in your fuzzer. In the examination version, maybe the program will crash for `typeflag=\x91`, maybe it will not crash for any value of `typeflag`.

Your fuzzer must be smart: you cannot try every value for every field. Trying all the values for the field name would imply formatting (2^8)100 archives, which is not sustainable. For example, you can soundly assume that if the name field accepts every non-ascii value at every position of the string, combining every string of some non-ascii characters is useless. This will allow your fuzzer to run in a reasonable time.

Your fuzzer must work with archives: you cannot just try different values for different fields in the header. You must deal with headers with and without data, with archives containing multiple files, etc.

Deliverable

Submission will be done on INGInious: https://inginious.info.ucl.ac.be/course/LINFO2347/project (the task will open soon).
You must register to a group on INGinious with your partner to submit.
You must upload a zip file before Friday, March 20th, 14:00, containing exactly:

- The source code of your fuzzer.
- A Makefile which compiles your project to an executable named *fuzzer*. The generated executable takes one argument: the path to the tar extractor.

You must implement your fuzzer in C, version 99 or later. The source code must be compilable with gcc, on a 64-bit x86-64 Linux system without any additional dependencies other than the standard libraries. If you don't own a computer with such architecture, you can use the machines in the student rooms. To test whether your fuzzer works correctly, we will copy the executable into a directory together with an extractor and start it from there. **Your fuzzer may not rely on any other file**. The input files must be generated in the same directory. We will assume that all generated files with a name starting with "success_" contain crashing input files.

Implementation hints

You do not have to perform fuzzing on the fields *devmajor*, *devminor*, *prefix* or *padding*.
The file *help.c* (in /home/student/fuzzer/) contains:

- The header structure you are supposed to use.
- An example of code that launches a program given as argument, parses its output and checks whether it matches the crash message "`*** The program has crashed ***`".
- A function that computes the checksum of a header and writes it in the *chksum* field.

AI guidelines

The use of generative AI for code production is prohibited in this project (and would be ineffective). In case of doubt, the teaching team reserves the right to conduct an oral assessment of the project to

ensure that all students in the group have mastered the content, in accordance with EPL regulations on the use of generative AI.